Bluetooth Controlled RC Car using ESP32

1. Overview

This project is a smartphone-controlled Remote Control (RC) car that utilizes an ESP32 microcontroller for wireless communication and logic processing. The system connects to a standard Android device via Classic Bluetooth. The user sends directional commands through a virtual Gamepad application, which the ESP32 interprets to drive a dual-motor setup via an L298N motor driver. The car features "Tank Turn" mechanics, allowing it to rotate on its own axis for high manoeuvrability.

2. Hardware Components

- **Microcontroller:** ESP32 Development Board (ESP-WROOM-32)
- **Motor Driver:** L298N Dual H-Bridge Motor Driver
- **Actuators:** 2x DC Gear Motors with Wheels
- **Power Supply:** Lithium-ion Battery Pack (e.g., 2x 18650 cells)
- **Chassis:** Standard 2-Wheel Drive (2WD) chassis with a front caster wheel
- **Connectors:** Jumper wires (Male-to-Female, Male-to-Male)

3. Circuit & Wiring Details

The ESP32 communicates with the L298N driver using standard GPIO pins for direction control and PWM (Pulse Width Modulation) for speed control.

ESP32 Pin	L298N Pin	Function
GPIO 13	IN1	Right Motor Forward
GPIO 12	IN2	Right Motor Backward
GPIO 14	IN3	Left Motor Forward
GPIO 27	IN4	Left Motor Backward
GPIO 26	ENA	Right Motor Speed (PWM)
GPIO 25	ENB	Left Motor Speed (PWM)
VIN	5V Out	Power supply to ESP32
GND	GND	Common Ground

Power Note: The L298N is directly powered by the battery via its 12V input. The L298N's internal voltage regulator steps this down to 5V, which is then fed into the ESP32's VIN pin to power the microcontroller.

4. Software & Application Setup

- **Development Environment:** Arduino IDE

- **Library Used:** BluetoothSerial.h (Native ESP32 Classic Bluetooth library)
- **Android Application:** "Arduino Bluetooth Controller" (Configured in Gamepad/Controller Mode)
- **Command Mapping:**
    - Forward: F
    - Backward: B
    - Left: L
    - Right: R
    - Stop / Break: S (Sent automatically on button release)

5. Movement Logic & Motor Calibration

To achieve straight-line movement despite hardware inconsistencies (where one motor may naturally spin faster than the other), PWM calibration is applied. In this build, the Left motor is scaled down slightly compared to the Right motor to maintain a perfect trajectory.

For turning, the system uses **Differential Steering (Tank Turn)**. To turn Left, the right motor moves forward while the left motor moves backward simultaneously. This provides a zero-radius turn, highly beneficial for tight spaces.

6. Source Code

C++

```
#include "BluetoothSerial.h"


BluetoothSerial SerialBT;


#define IN1 13

#define IN2 12

#define IN3 14

#define IN4 27

#define ENA 26

#define ENB 25


char command;


void setup() {

 Serial.begin(115200);
```

```
  SerialBT.begin("Robosoccer_ESP32");
  Serial.println("Bluetooth Started! Ready to pair...");

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);

  analogWrite(ENA, 205);
  analogWrite(ENB, 255);
}

void loop() {

  if (SerialBT.available()) {
    command = SerialBT.read();
    executeCommand(command);
  }
}

void executeCommand(char cmd) {
  switch (cmd) {
    case 'f':
      digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
      digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
      break;

    case 'B':
```

```
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
    break;

  case 'R':
    digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
    break;

  case 'L':
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, HIGH);
    break;

  case 'S':
  default:
    digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW); digitalWrite(IN4, LOW);
    break;
 }
}
```